Figure 12.3 The SSLEngine handshake state machine involves four main states. In the NEED_UNWRAP and NEED_UNWRAP_AGAIN states you should use the unwrap() call to supply it with received network data. The NEED_WRAP state indicates that new DTLS packets should be retrieved with the wrap() call and then sent to the other party. The NEED_TASK state is used when the engine needs to execute expensive cryptographic functions.

Listing 12.2 shows the outline of how the basic loop for performing a DTLS handshake with `SSLEngine` is performed based on the handshake status codes.

> **NOTE** You don't need to type in this example, because I have provided a wrapper class that hides some of this complexity and demonstrates correct use of the SSLEngine. See https://github.com/NeilMadden/apisecurityinaction/blob/chapter12/natter-api/src/main/java/com/manning/apisecurityinaction/DtlsDatagramChannel.java. You'll use that class in the example client and server shortly.

Listing 12.2 SSLEngine handshake loop

```
engine.beginHandshake();   #A

var handshakeStatus = engine.getHandshakeStatus();    #C
while (handshakeStatus != HandshakeStatus.FINISHED) {    #C
    SSLEngineResult result;
    switch (handshakeStatus) {
        case NEED_UNWRAP:    #D
```